DOKUMEN DESAIN ARSITEKTUR SMART CONTRACT

Sistem Verifikasi Sertifikat & Ijazah Digital Berbasis Blockchain

Mata Kuliah: Teknologi & Rekayasa Blockchain (Berbasis Proyek) | Dosen Pengampu: Mohamad Firdaus, M.Kom.

1. Pendahuluan & Latar Belakang

Sistem verifikasi ijazah dan sertifikat akademik konvensional saat ini masih menghadapi berbagai tantangan signifikan. Proses verifikasi manual yang dilakukan oleh instansi maupun pihak perusahaan (HRD) memakan waktu yang lama, memerlukan biaya administrasi, serta sangat bergantung pada otoritas terpusat. Di samping itu, tingkat pemalsuan dokumen fisik maupun dokumen digital dalam format PDF biasa semakin meningkat seiring dengan kemudahan manipulasi teknologi grafis.

Solusi berbasis Blockchain menawarkan pendekatan terdesentralisasi melalui pembuatan *Smart Contract* pada jaringan Ethereum/EVM. Sistem ini mencatat kredensial ijazah secara permanen (*immutable*) tanpa dapat diubah atau dimanipulasi oleh pihak mana pun setelah diterbitkan. Untuk mengoptimalkan penggunaan biaya transaksi (*Gas Fee*), file fisik PDF ijazah diolah menggunakan algoritma kriptografi menjadi bentuk *Cryptographic Hash* (misalnya SHA-256 atau IPFS Content Identifier/CID), sehingga hanya sidik jari digital dokumen yang disimpan di dalam rantai blok.

2. Desain Entitas (Struct, Enum, & Mapping)

Arsitektur data dalam Smart Contract dipetakan secara terstruktur untuk memodelkan objek dan status keabsahan dokumen akademik di dunia nyata:

2.1. Enum: StatusSertifikat

Enum digunakan untuk mengelola status siklus hidup dari sebuah ijazah/sertifikat:

- **Valid (0):** Menandakan sertifikat sah, terdaftar, dan diakui oleh instansi penerbit.
- **Dicabut (1):** Menandakan sertifikat telah dibatalkan atau dicabut keabsahannya (misalnya akibat pelanggaran akademik atau pembatalan gelar).

2.2. Struct: DataSertifikat

Struct memodelkan atribut utama dari dokumen ijazah:

- **namaPemilik (string):** Nama lengkap mahasiswa/penerima sertifikat.

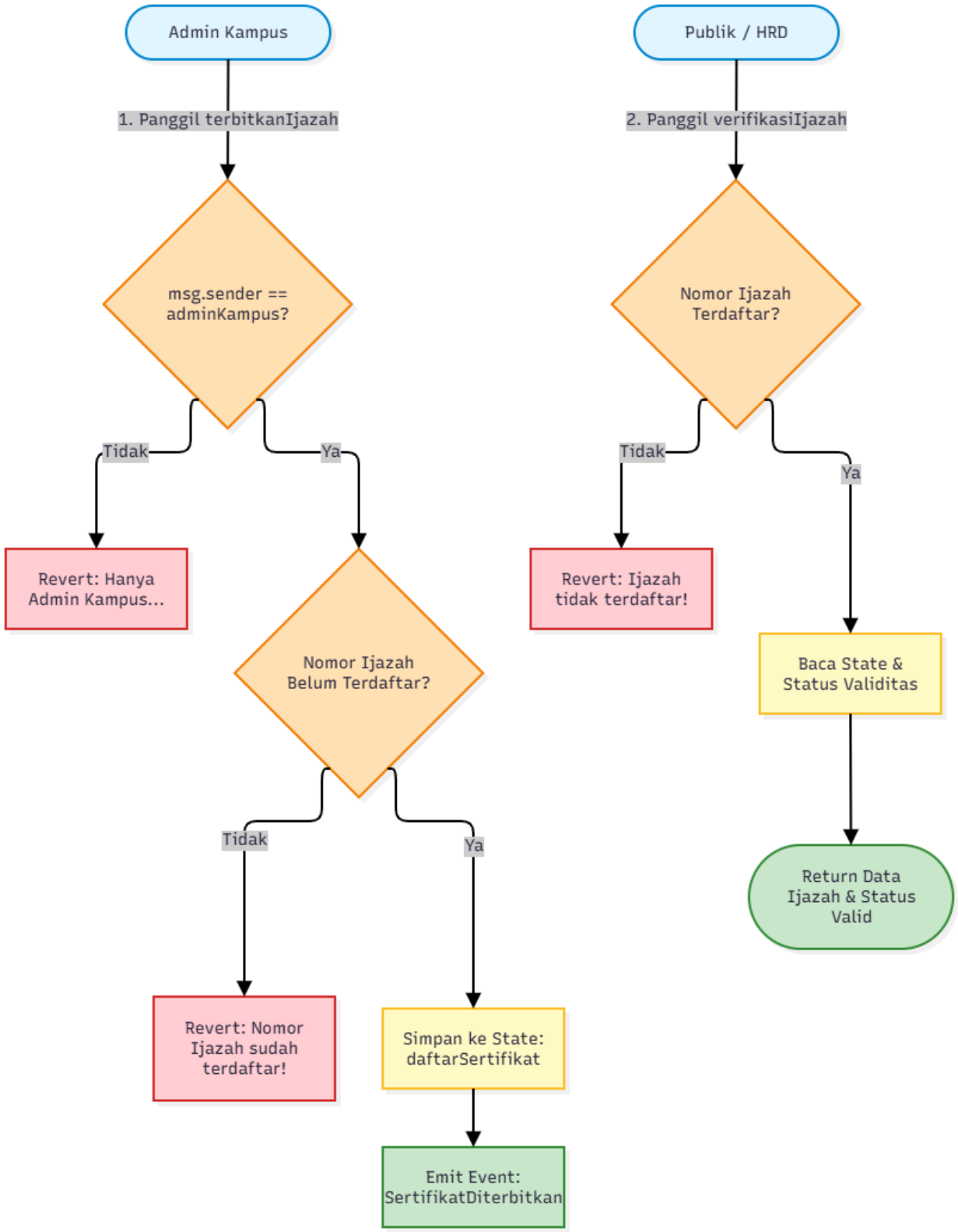
- **programStudi (string):** Program studi atau bidang keahlian penerima.
- **hashDokumen (string):** Hash kriptografi (IPFS CID / SHA-256) dari file ijazah asli.
- **tanggalTerbit (uint256):** Tanda waktu (*timestamp*) saat ijazah diterbitkan secara terenkripsi.
- **status (StatusSertifikat):** Status keabsahan dokumen (Valid / Dicabut).

2.3. Mapping: daftarSertifikat

Penyimpanan state utama menggunakan struktur data mapping(string => DataSertifikat). Fungsi ini mengasosiasikan **Nomor Ijazah Unik (string)** sebagai *Key* langsung ke objek **DataSertifikat (Struct)** sebagai *Value*. Pendekatan ini memberikan efisiensi pencarian data secara instan dengan kompleksitas waktu O(1).

3. Flowchart Alur Transaksi & Interaksi System

Interaksi pengguna dengan Smart Contract dibagi menjadi dua alur utama: Alur Penerbitan (Admin) dan Alur Verifikasi (Publik/HRD).


Admin Kampus
1. Panggil terbitkanIjazah
msg.sender == adminKampus?
Tidak
Ya
Revert: Hanya Admin Kampus...
Nomor Ijazah Belum Terdaftar?
Tidak
Ya
Revert: Nomor Ijazah sudah terdaftar!
Simpan ke State: daftarSertifikat
Emit Event: SertifikatDiterbitkan
Publik / HRD
2. Panggil verifikasiIjazah
Nomor Ijazah Terdaftar?
Tidak
Ya
Revert: Ijazah tidak terdaftar!
Baca State & Status Validitas
Return Data Ijazah & Status Valid

3.1. Deskripsi & Penjelasan Alur Sistem (Flowchart)

Gambar diagram di atas menggambarkan arsitektur alur kerja (workflow) Smart Contract yang dibagi menjadi dua skenario interaksi utama:

1. **Alur Penerbitan Ijazah (Aktor: Admin Kampus)**
   - **Inisiasi Transaksi:** Admin Kampus memanggil fungsi terbitkanIjazah() dengan menginputkan nomor ijazah, nama pemilik, program studi, dan *cryptographic hash* dokumen (IPFS CID/SHA-256).
   - **Validasi Otorisasi (Revert Check 1):** Sistem mengecek kondisi msg.sender == adminKampus melalui modifier onlyAdmin. Jika fungsi dipanggil oleh akun selain admin, transaksi otomatis dibatalkan (*Revert*) dengan pesan kesalahan "Error: Hanya Admin Kampus yang memiliki akses!".
   - **Validasi Duplikasi (Revert Check 2):** Jika otorisasi valid, sistem melakukan validasi apakah nomor ijazah tersebut sudah pernah terdaftar di variabel *mapping*. Jika sudah ada, transaksi di-*revert* untuk mencegah duplikasi data.
   - **Penyimpanan State & Logging:** Jika seluruh pengondisian require terpenuhi, data ijazah disimpan ke dalam *storage* blockchain (daftarSertifikat) dan sistem memancarkan *Event* SertifikatDiterbitkan sebagai pencatatan riwayat transaksi yang efisien secara *gas fee*.

2. **Alur Verifikasi Ijazah (Aktor: Publik / HRD)**
   - **Inisiasi Pencarian:** Pihak eksternal (masyarakat umum/HRD) memanggil fungsi *read-only* verifikasiIjazah() dengan memasukkan kueri parameter Nomor Ijazah.
   - **Pengecekan Keberadaan Data:** Sistem memeriksa keberadaan nomor ijazah pada *state mapping*. Jika data tidak ditemukan, sistem mengembalikan respon kesalahan (*Revert*).
   - **Output Verifikasi:** Jika nomor ijazah terdaftar, sistem membaca data dari rantai blok dan mengembalikan (*return*) detail informasi pemilik dokumen serta status keabsahan dalam bentuk nilai boolean (isValid = true) secara instan dan bebas biaya gas (*free gas*).

4. Kode Smart Contract (Solidity)

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.20;

/**
 * @title Sistem Verifikasi Sertifikat & Ijazah Digital
 * @notice Kontrak untuk menerbitkan dan memverifikasi keaslian ijazah/sertifikat akademik secara desentral.
 */
contract SertifikatIjazah {
    enum StatusSertifikat { Valid, Dicabut }
```

```
    struct DataSertifikat {
        string namaPemilik;
        string programStudi;
        string hashDokumen;
        uint256 tanggalTerbit;
        StatusSertifikat status;
    }

    address public adminKampus;
    mapping(string => DataSertifikat) public daftarSertifikat;

    event SertifikatDiterbitkan(string indexed nomorIjazah, string namaPemilik, string programStudi);
    event SertifikatDicabut(string indexed nomorIjazah, string alasan);

    modifier onlyAdmin() {
        require(msg.sender == adminKampus, "Error: Hanya Admin Kampus yang memiliki akses!");
        _;
    }

    constructor() {
        adminKampus = msg.sender;
    }

    function terbitkanIjazah(
        string memory _nomorIjazah,
        string memory _namaPemilik,
        string memory _programStudi,
        string memory _hashDokumen
    ) external onlyAdmin {
        require(
            bytes(daftarSertifikat[_nomorIjazah].namaPemilik).length == 0,
            "Error: Nomor Ijazah sudah terdaftar!"
        );
        require(bytes(_nomorIjazah).length > 0, "Error: Nomor Ijazah tidak boleh kosong!");

        daftarSertifikat[_nomorIjazah] = DataSertifikat({
            namaPemilik: _namaPemilik,
            programStudi: _programStudi,
            hashDokumen: _hashDokumen,
            tanggalTerbit: block.timestamp,
            status: StatusSertifikat.Valid
        });

        emit SertifikatDiterbitkan(_nomorIjazah, _namaPemilik, _programStudi);
    }

    function cabutIjazah(string memory _nomorIjazah, string memory _alasan) external onlyAdmin {
        require(
            bytes(daftarSertifikat[_nomorIjazah].namaPemilik).length > 0,
            "Error: Nomor Ijazah tidak ditemukan!"
        );
        require(
            daftarSertifikat[_nomorIjazah].status == StatusSertifikat.Valid,
            "Error: Ijazah sudah dalam status dicabut!"
        );
```

```
        daftarSertifikat[_nomorIjazah].status = StatusSertifikat.Dicabut;

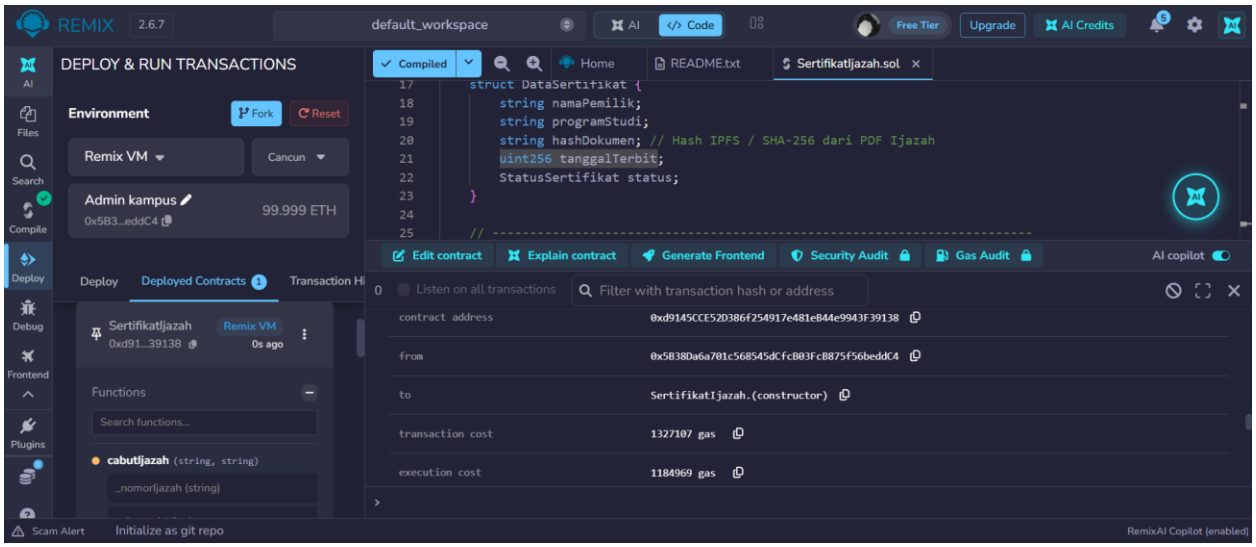
        emit SertifikatDicabut(_nomorIjazah, _alasan);
    }

    function verifikasiIjazah(string memory _nomorIjazah) external view returns (
        string memory namaPemilik,
        string memory programStudi,
        string memory hashDokumen,
        uint256 tanggalTerbit,
        bool isValid
    ) {
        require(
            bytes(daftarSertifikat[_nomorIjazah].namaPemilik).length > 0,
            "Error: Ijazah tidak terdaftar di sistem!"
        );

        DataSertifikat memory cert = daftarSertifikat[_nomorIjazah];
        bool statusValid = (cert.status == StatusSertifikat.Valid);

        return (
            cert.namaPemilik,
            cert.programStudi,
            cert.hashDokumen,
            cert.tanggalTerbit,
            statusValid
        );
    }
}
```

SCREENSHOT COMPILATION & DEPLOYMENT SUCCESS



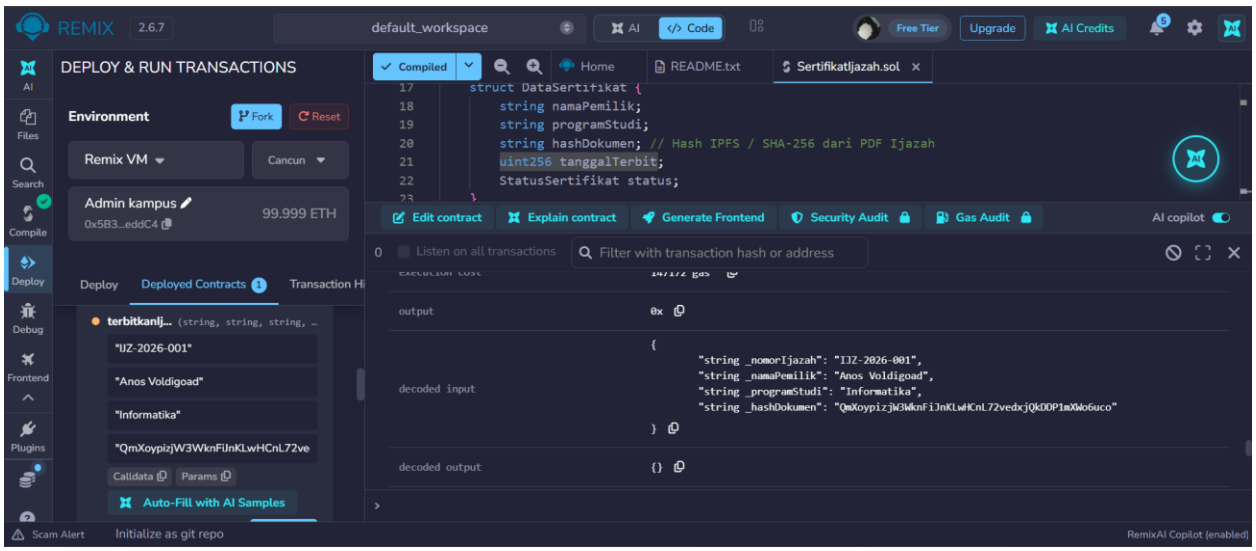
5. Hasil Pengujian & Simulasi (Live Demo)

5.1. Pengujian Alur Normal (Happy Path)

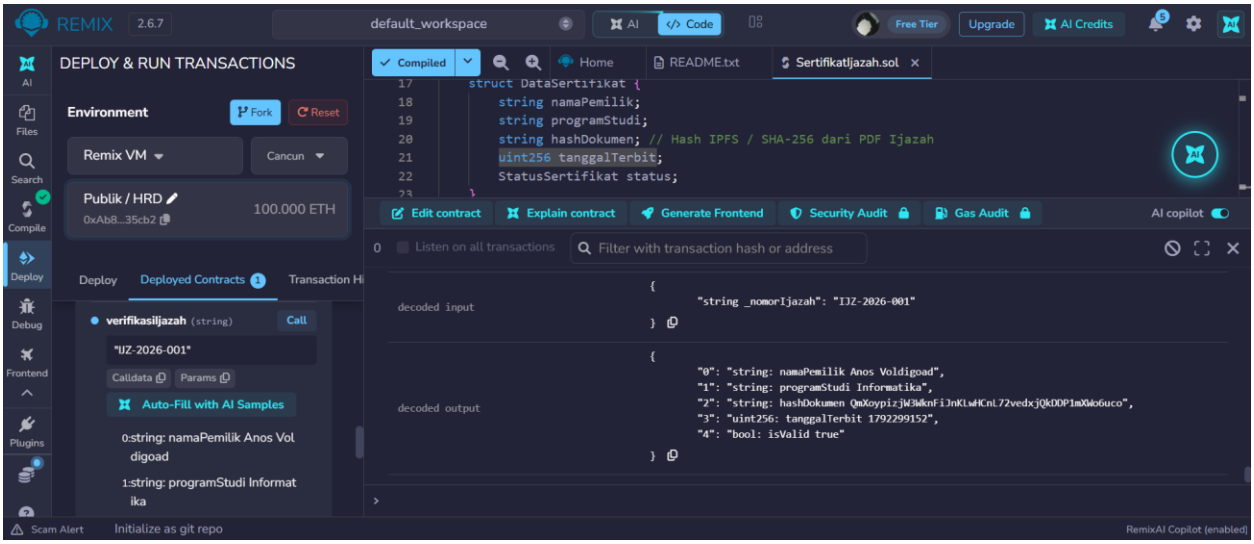
Pengujian dilakukan menggunakan Akun Admin (Account 1) untuk menerbitkan sertifikat baru dengan data sampel. Selanjutnya, Akun Publik/HRD (Account 2) memanggil fungsi verifikasiIjazah untuk membaca data.

- **Input Terbit:** Nomor Ijazah: "IJZ-2026-001", Nama: "Anos Voldigoad", Prodi: "Informatika", Hash: "QmXoypizjW3WknFiJnKLwHCnL72vedxjQkDDP1mXWo6uco".
- **Hasil Verifikasi:** Data berhasil diretur secara utuh dengan status boolean isValid = true.

SCREENSHOT HASIL PENERBITAN IJAZAH



SCREENSHOT HASIL VERIFIKASI / HAPPY PATH

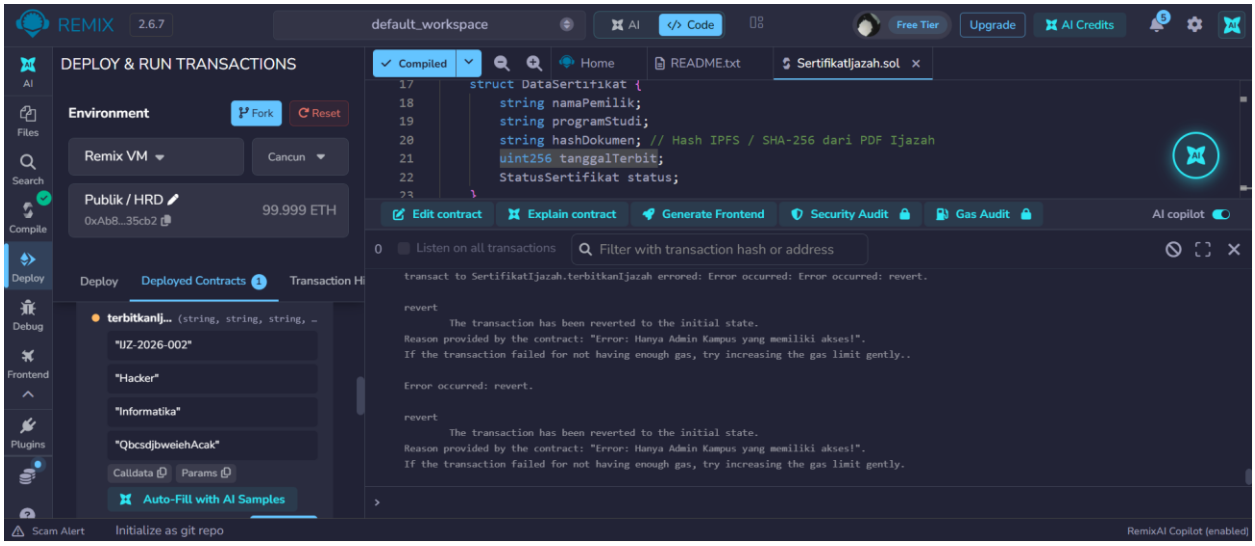


5.2. Pengujian Keamanan & Simulasi Serangan (Revert Check)

Simulasi serangan dilakukan dengan mengganti akun aktif menjadi Account 2 (Akun non-admin/penyerang). Penyerang mencoba memanggil fungsi terbitkanIjazah untuk memasukkan data ilegal.

Hasil Skenario Keamanan: Transaksi secara otomatis digagalkan oleh EVM (*Revert*) karena gagal melewati pengecekan modifier onlyAdmin. Terminal Remix menampilkan pesan kesalahan resmi dari smart contract.

SCREENSHOT SIMULASI SERANGAN / REVERT ERROR

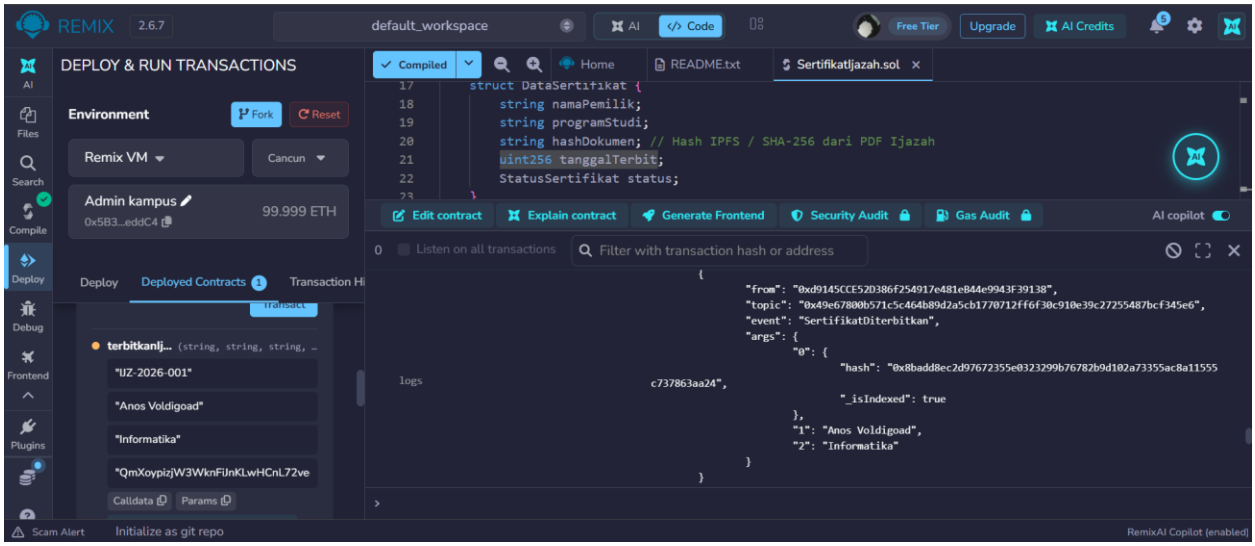


5.3. Pengujian Logging (Event Execution)

Penggunaan *Event* memfasilitasi pencatatan riwayat transaksi secara efisien di terminal blockchain tanpa

membebani memori variabel state secara permanen.

SCREENSHOT BUKTI LOGS / EVENT

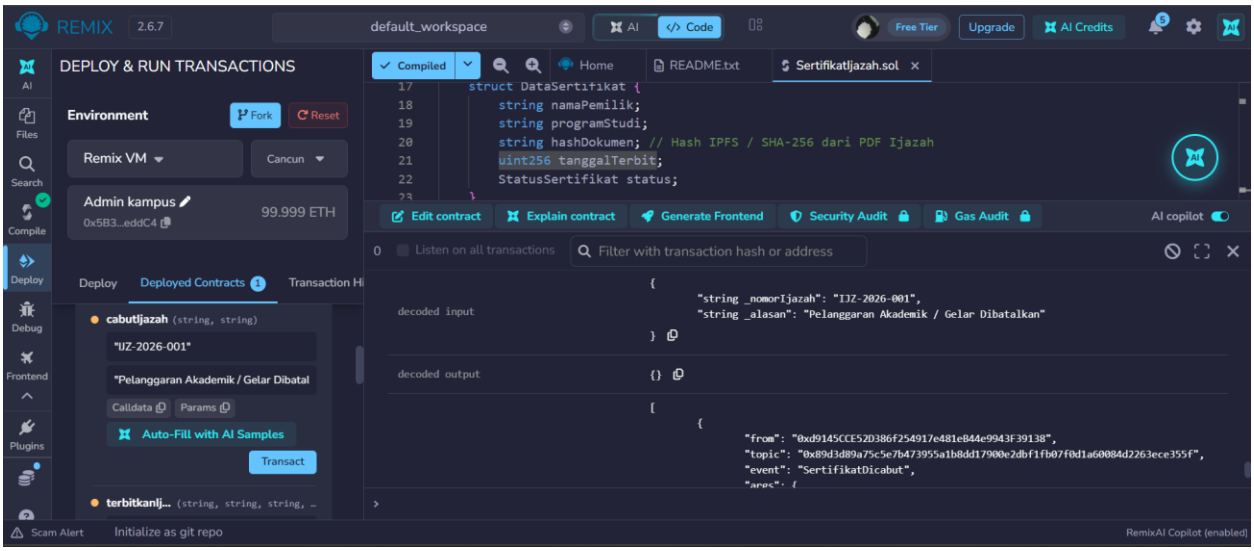


5.4. Pengujian Pencabutan Sertifikat (Cabut Ijazah)

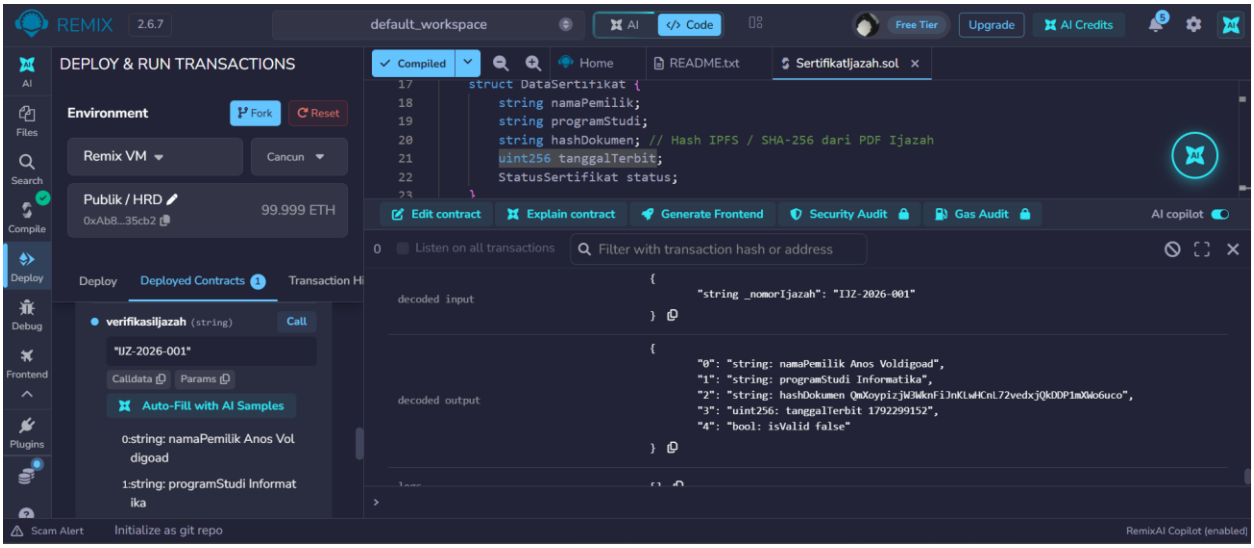
Pengujian pencabutan ijazah dilakukan oleh Admin Kampus (Akun 1) terhadap nomor ijazah "IJZ-2026-001" dengan memanggil fungsi cabutIjazah dan memberikan parameter alasan "Pelanggaran Akademik / Gelar Dibatalkan".

Transaksi berhasil dieksekusi di jaringan blockchain dan memancarkan *Event* SertifikatDicabut. Saat pihak HRD/Publik (Akun 2) kembali memanggil fungsi verifikasiIjazah("IJZ-2026-001"), sistem mengembalikan nilai isValid = false, yang menandakan bahwa status keabsahan dokumen ijazah tersebut telah resmi dicabut dan tidak lagi berlaku.

SCREENSHOT PROSES CABUT IJAZAH



SCREENSHOT HASIL VERIFIKASI SETELAH CABUT IJAZAH



6. Kesimpulan

Smart Contract versi 1.0 untuk verifikasi sertifikat dan ijazah digital ini telah memenuhi seluruh kriteria kelayakan fundamental Web3. Sistem terbukti aman dari potensi manipulasi data ilegal melalui proteksi otorisasi berbasis pendaftaran role admin, efisien dalam pemanfaatan gas fee berkat struktur event dan hash offline storage, serta siap diintegrasikan pada arsitektur dApp skala produksi.